Informação e Codificação - Miniprojeto 1

André Silva Dinis Oliveira Rafael Ferreira

October 2026

1 Introduction and Methodology

This report describes the audio tools (Part I, Section 2) and two codecs: a lossless one (Part II, Section 3) and a lossy DCT-based one (Part III, Section 4). The tools are written in Python and the codecs in C, for speed and low memory use. The usage of each program is given at the start of its section.

Requirements and build. The codecs need a C compiler and `make`: running `make` in the folder of each part builds the encoder and the decoder. The benchmarks also use GNU `time`, `flac`, `zstd`, `xz` and `ffmpeg` (with libmp3lame) as reference codecs.

Evaluation. Evaluations use 27 WAV files, all 16-bit stereo at 44.1 kHz, in three groups: the 7 course samples, 10 tracks from DSD100 and 10 from EBU SQAM (TECH3253).

2 Part I: Audio Tools

2.1 Sample histograms (`wav_hist`)

`wav_hist` reads a WAV file and saves one histogram image per channel and per value of `K`:

```
python3 wav_hist.py input.wav [--k K [K ...]]  [--outdir DIR]
```

Channels. The samples are read with Python's `wave` module into `numpy` arrays of signed integers. A mono file gives one histogram. For stereo input the program plots the left channel L, the right channel R and, with integer (floor) division as the assignment requires,

$$\text{MID} = \left\lfloor \frac{L+R}{2} \right\rfloor, \qquad \text{SIDE} = \left\lfloor \frac{L-R}{2} \right\rfloor.$$

Flooring both halves discards the parity of $L+R$, so these two plots alone cannot be inverted. Keeping $L-R$ undivided as the SIDE channel makes the pair invertible, because $L+R$ and $L-R$ always have the same parity; this is what the lossless codec uses (Section 3).

Coarser bins. With `--k K`, each value v is replaced by $\lfloor v/2^K \rfloor$ before counting, so each bin gathers 2^K consecutive values; $K = 0$ gives one bin per value. For 16-bit audio, $K = 6$ reduces the 65 536 possible values to 1 024 bins.

Results. Figure 1 shows the histograms of `sample01`. The MID histogram is bell-shaped and centred on zero, and SIDE is concentrated much closer to zero than L and MID, which is what makes mid/side coding useful (Section **??**). With bins of width 64, the irregular fluctuations between neighbouring values disappear and the shape of the distribution is clear.

2.2 Quantization (`wav_quant`)

`wav_quant` performs uniform scalar quantization and writes a WAV file:

```
python3 wav_quant.py input.wav output.wav (--bits B | --levels N)
```

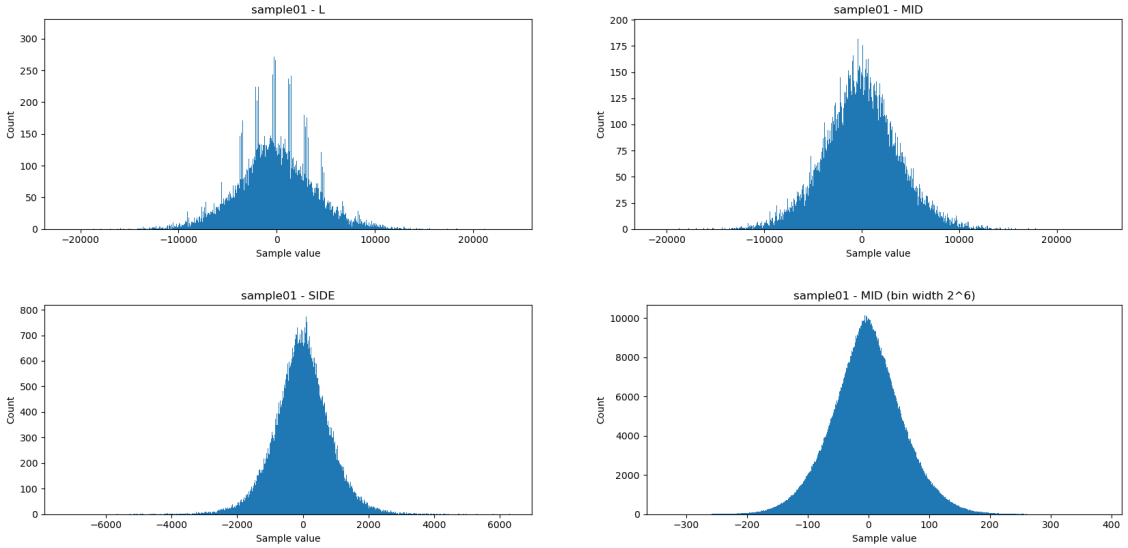


Figure 1: Histograms of `sample01`. Top: L and MID. Bottom: SIDE, and MID with bins of width $2^6 = 64$ (`--k 6`).

Method. For a file with b bits per sample, `--bits B` keeps 2^B levels, with step $\Delta = 2^{b-B}$ (for example $\Delta = 256$ for 8 of 16 bits). `--levels N` accepts any number of levels, not only powers of two, with $\Delta = \lfloor 2^b/N \rfloor$. Each sample x is replaced by the largest multiple of Δ not above it,

$$y = \left\lfloor \frac{x}{\Delta} \right\rfloor \Delta,$$

which for powers of two is a right shift followed by a left shift. The quantizer therefore truncates: the error $x - y$ always lies in $[0, \Delta - 1]$. Zero is always one of the levels, and y never leaves the original range, so no clipping is needed.

The output keeps the sample width of the input, so the file size does not change; what is reduced is the number of distinct values. That is what makes the signal cheaper to code afterwards.

Results. Figure 2 shows the effect on `sample01`. The histogram of the quantized output, with one bin per level (`--k 8` for 8 bits), has the same shape as the original, but the signal can only take 2^8 different values. The SNR for several bit depths is given with the `wav_cmp` results (Table 1).

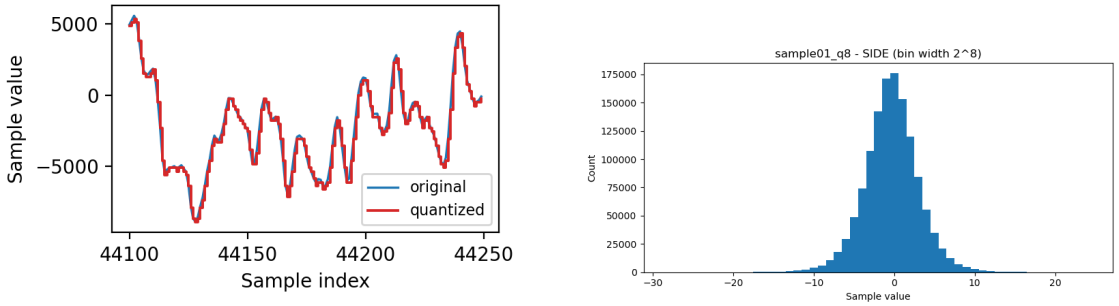


Figure 2: Effect of `wav_quant` on `sample01`. Left: original and quantized waveform over a short excerpt. Right: histogram of the quantized output, one bin per level.

2.3 Comparing files (`wav_cmp`)

`wav_cmp` compares a processed WAV file with its original and prints three error measures. We use it to measure the error of the other Part I tools and of the lossy codec.

```
python3 wav_cmp original.wav processed.wav
```

How it works. The program first checks that both files have the same sample rate, number of channels and bit depth, and stops with an error if they do not. It then reads both files at the same time and compares each sample $y_c(n)$ of the processed file with the sample $x_c(n)$ at the same position in the original. The error is $e_c(n) = x_c(n) - y_c(n)$, where c is the channel and n the position.

Error measures. The three measures are given for the left channel, the right channel and the average of the channels, which is the mono signal $(L + R)/2$. With N the number of samples of a channel:

- **Mean squared error** (MSE, the L^2 norm): the average of the squared error. It is the average power of the error, so it depends on how loud the file is.

$$\mathrm{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} e(n)^2$$

- **Maximum absolute error** (the L^∞ norm): the largest error of a single sample. A large value together with a high SNR points to a short problem that the averages hide, such as clipping.

$$E_{\max} = \max_n |e(n)|$$

- **Signal-to-noise ratio** (SNR): the energy of the original divided by the energy of the error, in decibels. A higher value means a smaller error. It does not depend on how loud the file is, so it can be compared between files, and it is the measure the lossy codec optimizes (Section 4).

$$\mathrm{SNR} = 10 \log_{10} \frac{\sum_n x(n)^2}{\sum_n e(n)^2} \ \mathrm{dB}$$

If the two files are identical, the error is zero, the SNR is infinite, and this is how `wav_cmp` confirms a lossless round trip. If the original is silent, the SNR is undefined and only the MSE and the maximum error are printed.

Results. Table 1 shows `wav_cmp` applied to `sample01.wav` quantized with `wav_quant` to 8 and to 10 bits (steps $\Delta = 256$ and $\Delta = 64$).

Quantized to	Channel	MSE	max $\lvert e \rvert$	SNR (dB)
8 bits	L	21708.5393	255	28.4201
	R	21716.4586	255	28.5210
	average	18920.0543	255	28.8501
10 bits	L	1331.8234	63	40.5419
	R	1333.2413	63	40.6398
	average	1146.2529	63	41.0265

Table 1: `wav_cmp` results for `sample01.wav` quantized to 8 and 10 bits.

The numbers agree with how `wav_quant` works:

- `wav_quant` truncates, so the error of each sample is between 0 and $\Delta - 1$. The maximum errors, 255 and 63, are exactly $\Delta - 1$.
- The error is spread evenly over that range, which gives MSE $\approx \Delta^2/3$: 21 845 and 1 365 expected, against about 21 710 and 1 332 measured.
- Two more bits raise the SNR from 28.4 to 40.5 dB: about 6 dB per bit.
- L and R differ by less than 0.2 dB. The average has a lower MSE and a higher SNR than both, because averaging two channels also averages their errors.

2.4 Audio effects (`wav_effects`)

`wav_effects` applies one effect to every channel of a WAV file:

```
python3 wav_effects.py input.wav output.wav EFFECT [options]
```

With sample rate f_s and input $x(n)$, the four effects are:

- **Echo** (`echo --delay T --gain g`): adds one delayed, quieter copy, $y(n) = x(n) + g\,x(n-D)$ with $D = \lfloor T f_s \rfloor$ samples.
- **Multiple echoes** (`multiecho --taps T1:g1 T2:g2 ...`): adds several delayed copies, $y(n) = x(n) + \sum_i g_i\,x(n - D_i)$.
- **Amplitude modulation** (`am --depth m --freq f`): the volume swings up and down, $y(n) = x(n)\,[1 + m\cos(2\pi f n / f_s)]$, with $0 \le m \le 1$.
- **Variable delay** (`vdelay --base d0 --depth a --lfo f`): mixes the signal with a copy whose delay changes smoothly, $y(n) = x(n) + x(n - d(n))$ with $d(n) = f_s\,[d_0 + a\sin(2\pi f n / f_s)]$. The delay is not a whole number of samples, so the copy is read by linear interpolation between the two nearest samples.

Design choices.

- The echoes are delayed copies of the *input*, not of the output. The effect is always stable and every echo is exactly where it was asked for. A feedback echo would give a long decaying tail with a single gain, but its gain would have to stay below 1 or the signal would grow without limit.
- If the result would clip, the output is scaled down to fit the sample range; otherwise it is left unchanged (normalization).
- The sum of a signal and its copies can pass the range of the sample format (amplitude modulation reaches $1+m$ times the input). The output is therefore scaled by (largest allowed value)/(largest value in the output) when it would clip, and is left unchanged otherwise.
- Samples before the start of the file are taken as zero.

Results. Figure 3 applies each effect to a short excerpt of sample02.wav. The echo adds a second attack 0.25 s after the first, and the multiple echoes add three weaker ones, 0.15, 0.30 and 0.45 s after the first. Amplitude modulation makes the envelope rise and fall at 4 Hz. In the variable delay, the signal and its moving copy add and cancel in turn, which changes the shape of the waveform.

3 Part II: Lossless Audio Codec

3.1 Method

```
wav_lossless_enc [-b N] [-v] input.wav compressed_file
wav_lossless_dec compressed_file output.wav
```

`-b` sets the block size in samples per channel (default 512) and `-v` prints statistics. The codec is written in C, because running time and memory are graded. It handles 8, 16 and 24-bit PCM, mono and stereo.

Each block of N samples goes through four steps: stereo transform → predictor → zigzag mapping → Rice coder.

Stereo. For stereo files the encoder codes MID $= \lfloor (L+R)/2 \rfloor$ and SIDE $= L - R$ instead of L and R. When the channels are similar, SIDE is small and cheap to code. The pair can be inverted exactly because $L + R$ and $L - R$ always have the same parity: $L + R = 2\,\text{MID} + (\text{SIDE} \bmod 2)$ (see also Section 2.1). SIDE needs one more bit than the input, which the codec allows for.

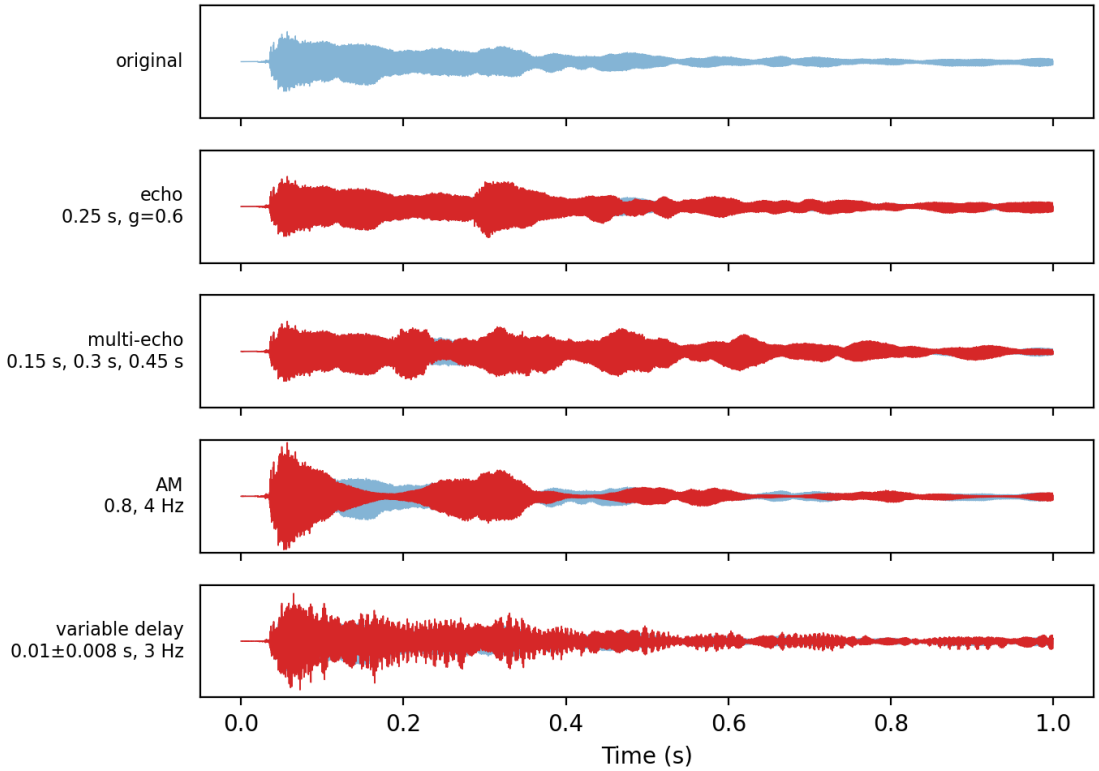


Figure 3: Effects on an excerpt of sample02.wav. Blue: original; red: result. All rows use the same vertical scale, and the final scaling of `wav_effects` is not applied.

Prediction. Each channel is predicted from its previous samples with a fixed polynomial predictor of order 0 to 4. The residual of order k is the k-th difference of the signal, for example $e(n) = x(n) - 2x(n-1) + x(n-2)$ for order 2. For each block and channel the encoder keeps the order with the smallest sum of $|e(n)|$, which is a cheap estimate of the coded size. Fixed predictors need no coefficients in the file and are very fast; adaptive linear prediction, as in `flac`, can compress more but costs more time (Section 3.3).

Entropy coding. Residuals are signed, so they are first mapped to non-negative numbers with the zigzag mapping, $u = 2e$ for $e \geq 0$ and $u = -2e - 1$ otherwise, so that small errors give small u. Each u is then written as a Rice code with parameter k: the quotient $\lfloor u/2^k \rfloor$ in unary (that many zeros and a one), followed by the k low bits of u. A value costs $\lfloor u/2^k \rfloor + 1 + k$ bits. Residuals are roughly Laplacian, for which a Rice code is close to optimal, and it needs no tables. The encoder picks k for each block and channel by computing the exact cost of a few values around $\log_2$ of the mean.

3.2 File Format and Exact Reconstruction

The decoder must reproduce the input byte for byte, not only its samples. The encoder therefore splits the input into three parts: the bytes before the first sample (the WAV header), the audio frames, and the bytes after them (extra chunks, padding, a partial last frame). The first and last part are stored unchanged in the compressed file, and the decoder joins prefix, decoded samples and suffix again.

The compressed file starts with a small header (a 4-byte identifier, mode, channels, bit depth, block size, number of frames and the lengths of the stored prefix and suffix). It is followed, for each block and channel, by a record with the predictor order (3 bits) and the Rice parameter (5 bits), and the Rice-coded residuals. A file that is not a supported WAV (another format, 32-bit or float samples, more than two channels, damaged header) is stored unchanged in a second, raw mode, so every input still round-trips.

The decoder checks the header against the size of the compressed file and rejects inconsistent files. Testing covered 55 synthetic cases (all bit depths, mono and stereo, files of 0 to several blocks, extra chunks) and 60 damaged files, with sanitizers: every decoded file was identical to its input, and no damaged file crashed the decoder. All files in the evaluation below were also reproduced exactly.

3.3 Evaluation

We compare our codec with `flac` and the general-purpose compressors `xz`, `zstd`, `bzip2` and `gzip`, all single-threaded like ours, on the 27 files (150 MB, 14 min of audio, largest file 13 MB). Table 2 gives totals: the ratio is total original size over total compressed size, times are sums of per-file medians over 3 runs, and peak memory is the maximum over the files. Every decoded file was byte-identical to its original, for every program.

			Speed (MB/s)		Peak memory (MiB)	
Program	Ratio	Bits/sample	Enc.	Dec.	Enc.	Dec.
Ours (block 512)	2.164	7.39	136.5	154.6	38.5	26.2
`flac -8`	**2.391**	**6.69**	48.4	161.7	3.5	3.0
`flac -5`	2.364	6.77	100.1	165.3	3.5	3.0
`xz -9e`	1.822	8.78	5.7	31.4	178.8	14.5
`zstd -19`	1.646	9.72	9.9	401.1	101.2	12.5
`zstd -3`	1.421	11.26	203.6	441.7	23.2	6.6

Table 2: Lossless compression of the 27 files (totals). `bzip2` and `gzip` (ratios 1.43 and 1.31) are in `bench_out/results.csv`.

Compression ratio. `flac -8` is the best audio codec at 2.39. Our codec reaches 2.16, so its files are 10.5% larger than `flac`'s, because we use only fixed predictors of order 0 to 4 and one Rice parameter per block, while `flac` fits a linear predictor to each block and uses several Rice parameters per block. Our files are 16% smaller than those of `xz -9e` (1.82), 24% smaller than those of `zstd -19` (1.65) and 34% smaller than those of `zstd -3` (1.42). More effort does not help the general-purpose tools much: `xz -6` and `xz -9e` give the same ratio, and `zstd -19` is 16% better than `zstd -3` but 21 times slower to encode.

The result depends on the kind of audio. On music, the audio codecs are far ahead of the general-purpose tools: ratios of 1.68 (ours) and 1.79 (`flac -8`) on the course samples, and 1.77 and 1.93 on DSD100, against 1.20 and 1.31 for `xz -9e`. General-purpose tools look for repeated byte strings, which recorded music almost never contains, while prediction turns correlated samples into small residuals. On the EBU SQAM test signals the picture changes: `xz -9e` reaches 3.45, slightly above `flac -8` (3.40), and `zstd -19` 2.98, against 2.95 for our codec, which is 15% larger than `flac -8` there (6% and 9% on the other two groups).

Between 10% and 24% of the samples of every SQAM file are exactly zero, so these files contain stretches of digital silence. A Rice code spends at least one bit on each sample, while `flac` stores a silent block as a constant and `xz` stores a long run of zeros in a few bytes. This explains only a part of the gap, at most a few percent of the file size; the rest comes from the better predictor of `flac` on the tonal, clean signals of this group.

Speed. Our codec encodes at 137 MB/s and decodes at 155 MB/s: 24 times faster than `xz -9e` to encode and 5 times faster to decode. Only `zstd -3` is faster (204 and 442 MB/s), and its files are 52% larger.

Memory. Our encoder needs 38.5 MiB for the largest file (13 MB) and our decoder 26.2 MiB: less than `xz -9e` (179 MiB) and `zstd -19` (101 MiB), more than `zstd -3` (23 MiB), and 11 times more than `flac` (3.5 MiB), which works one block at a time. Our codec keeps the whole file in memory: the encoder holds the input and the samples as 32-bit integers (twice the size for 16-bit audio) at the same time, so its peak grows by 3.0 MiB per MiB of input; the decoder holds the integer samples, 2.0 MiB per MiB. Memory is therefore not a problem for files of this size: the 27 files are at most 13 MB, so the peak stays below 40 MiB. A file of several hundred MB would need over a GB, which is the cost of not streaming.

Block size. Table 3 shows the ratio of our codec for each block size. Blocks of 256 and 512 samples compress best; the ratio falls steadily above that, from 2.16 to 2.10 at 8192. Large blocks force one predictor and one Rice parameter on a long stretch of changing audio, and a block header costs only 8 bits per channel, so small blocks lose little. Encoding and decoding times differ by less than 9% over the whole range. We use 512, which is the best or within 0.1% of the best in every group (2.3% smaller files than 4096 on DSD100).

Block size	All	Course	DSD100	SQAM
256	2.164	1.677	**1.771**	2.941
512	2.164	**1.678**	1.770	**2.945**
1024	2.160	1.677	1.765	2.941
2048	2.150	1.673	1.752	2.930
4096	2.129	1.665	1.729	2.904
8192	2.101	1.652	1.703	2.865

Table 3: Compression ratio of our codec for each block size, in total and per group.

4 Part III: Lossy Audio Codec

The lossy codec (`wav_lossy_enc` / `wav_lossy_dec`) aims at the best rate–distortion trade-off, with distortion measured as the signal-to-noise ratio

$$\mathrm{SNR} = 10 \log_{10} \frac{\sum x^2}{\sum (x-y)^2} \tag{1}$$

over all channels and samples of the original x and the decoded y (integers). Because this is a pure mean-squared-error measure, the codec uses no psychoacoustic model: every error counts the same, audible or not. This one decision shapes most of the design and explains the comparison with MP3 and AAC (Section 4.8). Like the lossless codec, it is written in C, since running time and memory are part of the assessment.

4.1 Design Overview

The encoder (i) reads the WAV file (8, 16 or 24 bits, mono or stereo); (ii) optionally converts left/right to mid/side; (iii) cuts each channel into blocks and transforms them; (iv) chooses one quantization step, either given (`--step`) or searched for a target SNR (`--snr`, default 30 dB); and (v) quantizes all coefficients with that step and codes them with a context-adaptive binary range coder. The decoder takes no options: it reads a 28-byte header, decodes and dequantizes the coefficients, inverts the transform and the stereo conversion, rounds and clips to the original bit depth, and writes a plain WAV file with the original rate, channels and bit depth.

The code is split into one module per block of this pipeline (transform, quantizer, rate control, coefficient coder, range coder), so that, for example, only the coefficient coder knows the statistics of the coefficients. With `--snr`, the encoder keeps all coefficients in memory as 32-bit floats, because the step search reads them many times; the decoder holds only the compressed file and one block per channel. The codec is checked by unit tests of each module, an end-to-end test (412 encode/decode runs over all bit depths, channel counts, block sizes, file lengths from 0 frames, and damaged or truncated files), and a build with address and undefined-behaviour sanitizers.

4.2 Transform

Two transforms are available: the block DCT-II, and the MDCT (the default), a lapped transform built on the DCT-IV. Both are orthonormal:

$$\text{DCT-II:}\quad X[k] = \sqrt{\tfrac{2}{N}}\, c_k \sum_{n=0}^{N-1} x[n] \cos\Big(\tfrac{\pi}{N}\big(n+\tfrac{1}{2}\big)k\Big), \qquad c_0 = \tfrac{1}{\sqrt{2}},\ c_{k>0} = 1, \tag{2}$$

$$\text{MDCT:}\quad X[k] = \sqrt{\tfrac{2}{N}} \sum_{n=0}^{2N-1} w[n]\, x[n] \cos\Big(\tfrac{\pi}{N}\big(n+\tfrac{1}{2}+\tfrac{N}{2}\big)\big(k+\tfrac{1}{2}\big)\Big), \qquad w[n] = \sin\Big(\tfrac{\pi(n+1/2)}{2N}\Big). \tag{3}$$

The MDCT reads $2N$ samples per block (N of them shared with the previous block) and produces N coefficients; the sine window and the overlap-add of consecutive blocks cancel the time-domain aliasing, so reconstruction is perfect without the discontinuities that the DCT has at block edges.

Orthonormality is what makes the rest of the design simple. By Parseval, the squared error of the coefficients equals the squared error of the samples, so (i) every coefficient's error counts equally towards the SNR, which makes one quantization step for everything the right choice (Section 4.4), and (ii) the encoder can predict the SNR from the coefficients alone, without running the inverse transform.

Both transforms are computed with a radix-2 complex FFT (so N is a power of two, 64 to 8192, default 2048): the DCT-II as an N-point FFT of a reordered input followed by a rotation, and the MDCT by folding the $2N$ windowed samples into N and computing a DCT-IV with an $N/2$-point FFT. The direct $O(N^2)$ formulas are kept as the reference for the tests; at $N = 2048$ the FFT versions are 736 (DCT) and 2474 (MDCT) times faster.

4.3 Stereo Coding

Stereo input can be coded as left/right, or as $M = (L + R)/2$, $S = (L - R)/2$ (default), with $L = M + S$, $R = M - S$. When the channels are similar, S is small and cheap. An error e_M, e_S becomes $e_M \pm e_S$ in L and R, so the total squared error is $2(e_M^2 + e_S^2)$: M and S have equal weight, one common step remains optimal, and the rate control multiplies the coefficient error by this weight. Each channel has its own probability models, since M and S have very different statistics.

4.4 Quantization and Rate Control

All coefficients of all blocks and channels share one uniform quantizer:

$$q = \operatorname{sign}(c) \left\lfloor \frac{|c|}{\Delta} + R \right\rfloor, \qquad c' = q\,\Delta. \tag{4}$$

With MSE as the measure and an orthonormal transform, an equal step everywhere gives every coefficient the same distortion–rate slope, which is the optimal allocation. The rounding offset $R = 0.4$ (instead of plain rounding, 0.5) creates a small dead zone: values just above a boundary go to zero, which the entropy coder represents very cheaply (Experiment 6). The decoder only computes $q\Delta$, so R is an encoder-only choice. The step is stored as a 32-bit float, and the encoder rounds it to that float before using it.

With `--snr`, the encoder looks for the *largest* step that reaches the target, which is the cheapest file that meets it:

1. a first guess from the uniform-noise model ($\Delta^2/12$ per sample);
2. a bracket around it, then 24 bisection rounds on $\log \Delta$, each estimating the SNR from the stored coefficients by Parseval (one quantize and sum pass, no inverse transform);
3. a correction for the decoder's rounding to integers, which the coefficient domain cannot see: with a per-sample error variance σ^2 before rounding, the error after it is $E[\mathrm{round}(e)^2]$ for $e \sim \mathcal{N}(0, \sigma^2)$, about $\sigma^2 + 1/12$ for large errors and close to 0 when $\sigma \ll 1$;
4. a verification by a real decode (the decoder's own code), shrinking the step by 0.5% if the target is missed, or raising it if it is exceeded by more than 0.1 dB.

In the 1080 encodes of the benchmark (27 files, 10 targets from 10 to 70 dB, four configurations), every target was reached and none was exceeded by more than 0.1 dB. Without step 3, targets above about 60 dB were missed by up to 0.33 dB after 20 slow verification decodes. The step is kept between a lower bound that keeps $|q| \le 2^{30}$ for any input and an upper bound above which every coefficient is zero.

4.5 Entropy Coding

The quantized values are coded with an adaptive binary range coder in the style of LZMA (32-bit range, 11-bit probabilities, adaptation by $1/2^5$ of the distance after each bit, carries through a cache byte). On random bits with known probabilities it comes within 0.4% of the entropy.

For each block and channel, the encoder first codes *nzb*, the number of 16-coefficient groups up to the last nonzero coefficient, with an adaptive bit tree; the run of zeros at high frequencies, where most coefficients end up, is then never coded. Each coefficient $q[k]$ before that point is binarized as $q = 0$?, then a sign bit (probability 1/2), then unary decisions $|q| > 1$?, …, $|q| > 14$?, and for $|q| \ge 15$ an Exp-Golomb code of $|q| - 15$.

The model of the zero flag and of the unary decisions is selected by a neighbourhood context computed from values the decoder already has, so no side information is sent:

$$m = 2|q[k-1]| + |q[k-2]| + |p[k-1]| + 2|p[k]| + |p[k+1]|, \tag{5}$$

where p is the previous block of the same channel, mapped to 8 classes $(0, 1, 2, 3\text{–}4, 5\text{–}6, 7\text{–}9, 10\text{–}14, 15+)$. It captures that large coefficients cluster in frequency (around tonal components) and in time (a tone lasts several blocks). A frequency-band context was also implemented but adds nothing once the neighbours are known (Experiment 4), so it is off by default.

4.6 File Format

A 28-byte little-endian header holds the magic `ICD1`, the number of channels, the bits per sample, the transform, the stereo mode, the sample rate, the number of frames (64 bits), N and the step (float32); the range-coder payload follows, block by block and channel by channel. The decoder rejects any header value the encoder cannot produce and any payload that ends early or has bytes left over (the decoder reads exactly what the encoder wrote), and removes its partial output on error. In the tests, every truncation, every impossible header value and all random bit-flip trials were rejected without a crash.

4.7 Experiments

Every design choice was tested on 27 files (16-bit stereo, 44.1 kHz, 14 min in total): the 7 course samples, the 10 DS100 mixes and the 10 SQAM test tracks. Each experiment changes one parameter from the default; for every configuration the encoder runs at targets of 15 to 45 dB on every file, the decoded files are measured independently, and the bitrate needed for 20, 30 and 40 dB is interpolated on each file's rate–distortion curve and averaged. Table 4 summarizes the results.

Table 4: Average bitrate (kbps) needed for a given SNR over the 27 files, for each change from the default. Lower is better; the last column is the change at 30 dB.

Experiment	Setting	20 dB	30 dB	40 dB	Δ @30 dB
Default (MDCT, N=2048, M/S, neighbours, shift 5, R=0.4)		**29.5**	**79.3**	**152.1**	–
1. Transform	DCT-II	32.8	85.6	161.2	+7.9%
2. Block size	$N = 256$	35.1	86.9	161.9	+9.6%
	$N = 512$	32.1	82.3	155.5	+3.8%
	$N = 1024$	30.0	79.1	151.2	−0.3%
	$N = 4096$	31.2	83.8	158.6	+5.7%
	$N = 8192$	34.5	90.8	168.7	+14.5%
3. Stereo	L/R	32.1	84.7	161.4	+6.8%
4. Contexts	none	32.2	83.5	156.8	+5.3%
	band only	31.8	82.9	156.1	+4.5%
	band + neighbours	29.6	79.4	152.0	+0.1%
5. Adaptation shift	4 (faster)	29.6	79.7	153.0	+0.5%
	6 (slower)	29.7	79.8	152.9	+0.6%
6. Rounding offset	$R = 0.5$	31.2	81.0	152.8	+2.1%
	$R = 0.45$	30.0	79.5	151.5	+0.3%
	$R = 0.35$	29.5	80.1	153.9	+1.0%
	$R = 0.3$	29.9	81.5	156.7	+2.8%

Transform. The MDCT needs fewer bits on 22 of the 27 files. The gain depends strongly on the material: 1.5% on the DS100 mixes, 5% on the course samples, but 37% on SQAM (18.8 against 29.7 kbps at 30 dB). SQAM is mostly solo tonal instruments, where the DCT's block edges and its rectangular window spread each tone over many coefficients.

Block size. Small blocks have poor frequency resolution; large blocks smear transients (and the quantization noise) over the whole block. The optimum depends on the material: 1024 for DS100 (111.1 against 114.2 kbps), 2048 for SQAM (18.8 against 20.8). Over all files, 1024 and 2048 are equivalent; we kept 2048 rather than tuning N to one dataset. Choosing N per file at encoding time (the best of 1024, 2048 and 4096) would save about 3%, but triples the encoding time, so we did not adopt it.

Stereo. Mid/side saves 6% on DS100 and 24% on SQAM, and wins on 23 of the 27 files. It loses when the channels are weakly correlated: on `sample02`, a wide stereo recording, it costs 17% more, since M and S then both mix two different signals.

Contexts and adaptation. The neighbourhood alone gives the whole context gain (5% at 30 dB, 8% at 20 dB); the frequency band adds nothing on top, because busy neighbourhoods are the low frequencies anyway. The gain shrinks at high SNR, where large coefficients dominate and their low bits are nearly random. The adaptation speed barely matters ($< 1\%$); 5 is best.

Rounding offset. $R = 0.4$ is the best (or tied best) setting at 20 and 30 dB on each of the three datasets separately (for example 114.2 against 116.6 kbps on DS100 at 30 dB), saving 2–6% over plain rounding. A larger dead zone ($R = 0.3$) zeroes too many coefficients and costs more at high SNR, where few coefficients are near zero.

4.8 Comparison with MP3 and AAC

MP3 (LAME) and AAC (FFmpeg's native encoder) were run through FFmpeg with one thread, like our codec, at 32 to 320 kbps, and decoded back to WAV. The SNR of every result was computed with our tool, which first searches for a decoder delay of up to 4096 frames (it was zero in every case, as FFmpeg compensates it). Time and peak memory were measured with GNU `time`, one file at a time. Figure 4 shows the average rate–distortion curves and Table 5 the main figures.

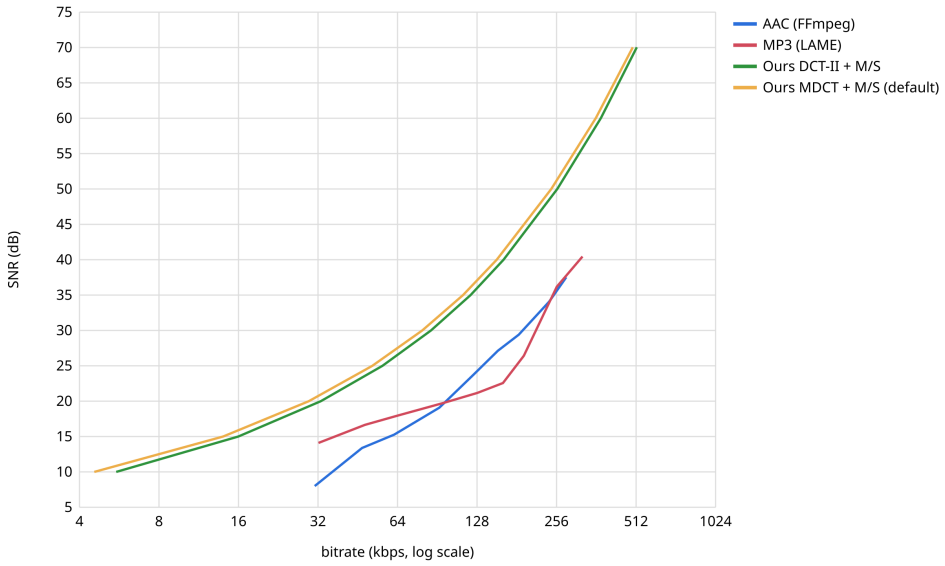


Figure 4: Rate–distortion curves over the 27 files: one point per encoder setting (SNR target for ours, bitrate for MP3/AAC) at the mean bitrate and mean SNR.

Measured by SNR, our codec is far better: on the same files, MP3 needs 3.4 times our bitrate for 20 dB and 3.3 times for 30 dB, and AAC 3.9 and 3.3 times; at 128 kbps we reach 41 dB against 21 dB (MP3) and 25 dB (AAC). This does not make it a better audio codec. MP3 and AAC are designed for perceived quality: their psychoacoustic models put noise where it is masked (under loud components, at high frequencies, after transients) and low-pass the signal, which costs a lot of SNR but is inaudible. Our codec does the opposite, spreading the noise evenly over frequency and time, which is optimal for MSE but more audible in quiet passages. Listening to the difference signals shows it directly: our error sounds like hiss that follows the music, while MP3's error at 128 kbps, at about the same level as ours at 20 dB, sounds like a muffled copy of the music itself. The comparison at equal SNR is the one the assignment asks for, but it says nothing about which codec sounds better.

Table 5: Comparison over the 27 files. Bitrate and compression ratio (1411.2 kbps / bitrate) at equal SNR; time per second of audio; peak memory. Where a codec's range does not cover a point on every file (MP3 and AAC within 32–320 kbps; one near-silent SQAM track never needs 128 kbps with ours), the average uses the files it covers (number in brackets).

	20 dB		30 dB		SNR @	Enc./dec.	Memory
Codec	kbps	ratio	kbps	ratio	128 kbps	(ms/s)	enc./dec. (MB)
Ours (MDCT, M/S)	29.5	47.8	79.3	17.8	41.1 dB (26)	18.4 / 4.8	40 / 5
Ours (DCT-II, M/S)	32.8	43.0	85.6	16.5	39.2 dB (26)	19.2 / 5.5	40 / 6
MP3 (LAME)	109.9 (24)	12.8	231.0 (25)	6.1	21.2 dB	19.9 / 4.3	55 / 54
AAC (FFmpeg)	115.4	12.2	190.5 (22)	7.4	24.7 dB	32.1 / 4.2	55 / 55

Our encoder is as fast as MP3 and faster than AAC (about 50 times real time, including the step search), and decodes about 200 times faster than real time. Its peak memory grows with the file (the input plus 4 bytes per sample for the stored coefficients; 40 MB for the longest, 74 s file), but stays below FFmpeg's, and the decoder needs only about 5 MB.

5 Discussion and Conclusions

Our lossless codec compresses the 27 files by a factor of 2.16, 16% better than `xz -9e` and 10.5% worse than `flac -8`, while encoding 2.8 times faster than `flac -8`. Prediction pays off on music, where repeated byte strings are rare; on the clean SQAM signals the general-purpose `xz` catches up. The gap to `flac` comes from design choices we made for speed and simplicity: fixed predictors and one Rice parameter per block.

Our lossy codec reaches 30 dB SNR at 79 kbps on average over the 27 files, a compression ratio of 17.8, and meets every requested SNR target to within 0.1 dB. At equal SNR, MP3 and AAC need 3.3 to 3.9 times its bitrate, while it encodes as fast as MP3 and faster than AAC. This advantage comes from optimizing exactly the measure being evaluated: with orthonormal transforms and one global quantization step, the bits go where they reduce the mean squared error, whereas MP3 and AAC spend them where the ear notices, so they sound better at the same SNR. The remaining losses come from choices we made for speed and simplicity: one block size for every file, where choosing it per file would save about 3% at three times the encoding time, and one stereo mode per file, which costs up to 17% on the few recordings whose channels are weakly correlated.